

ผู้ช่วยให้เหตุผลบนอุปกรณ์ปลายทาง ด้วย กราฟความรู้ที่ตรวจสอบได้

อธิบายวิธีคิดได้ ไม่สร้างไวยากรณ์ผิด และเก่งขึ้นโดย ไม่ต้องฝึกโมเดล

An On-Device Reasoning Assistant with a Verified Knowledge Graph:
it explains its reasoning, never hallucinates syntax, and improves without
retraining the model

ความเชื่อมโยงกับกลุ่ม 4: ระบบอัตโนมัติ/อุตสาหกรรมอัจฉริยะ · ประสิทธิภาพและการ
ใช้พลังงาน · การประยุกต์บนอุปกรณ์ปลายทาง

1. ปัญหาจริงที่เกิดขึ้นแล้วในวันนี้ (The Hook)

ผู้ช่วย AI เขียนโปรแกรมกำลังถูกใช้งานอย่างกว้างขวาง แต่หลักฐานเชิงประจักษ์
ปี 2025 ชี้ว่ามันสร้าง **ปัญหาจริงสามด้าน**พร้อมกัน:

- **เชื่อถือไม่ได้** — โค้ดที่ AI เขียน “ผิด/ไม่ปลอดภัย” ในอัตราสูง. รายงาน GenAI Code Security ของ Veracode ปี 2025 ทดสอบโมเดลภาษากว่า 100 ตัว พบว่า **45%** ของโค้ดที่ AI สร้างมีช่องโหว่ความปลอดภัย และมีช่องโหว่มากกว่าโค้ดที่มนุษย์เขียน **2.74 เท่า**¹ ยิ่งไปกว่านั้น งานวิจัยของ Stanford พบว่านักพัฒนาที่ใช้ AI ช่วย “เขียนโค้ดที่ปลอดภัยน้อยลง แต่กลับ มั่นใจมากขึ้น” ในความถูกต้องของมัน² — โมเดล แต่งไวยากรณ์/ตรรกะที่ผิดออกมาอย่างมั่นใจ (hallucination)
- **ความลับรั่วไหล** — โค้ดต้องถูกส่งออกนอกองค์กร. ปี 2023 วิศวกร Samsung นำซอร์สโค้ดลับไปวางใน ChatGPT เพื่อหาทางแก้บั๊ก เกิดการรั่วไหล **3 ครั้งใน 20 วัน** จนบริษัทต้อง **สั่งห้ามใช้ AI ทั้งองค์กร**³ เพราะข้อมูลถูกเก็บบนเซิร์ฟเวอร์ของผู้ให้บริการโดยลบไม่ได้

- **กินพลังงานมหาศาล — การพึ่งคลาวด์ไม่ยั่งยืน.** IEA รายงานว่าการใช้ไฟฟ้าของศูนย์ข้อมูล AI พุ่งขึ้น **50% ในปี 2025** และการใช้ไฟฟ้าของศูนย์ข้อมูลรวมจะ **เพิ่มเป็นสองเท่า** จาก 485 เป็น 950 TWh ภายในปี 2030⁴

สรุป: เครื่องมือที่มีอยู่บังคับให้ผู้ใช้เลือกระหว่าง ความสามารถ (คลาวด์) กับ ความเป็นส่วนตัว/ต้นทุน และไม่ว่าเลือกทางใด ก็ยัง ตรวจสอบไม่ได้ว่า AI คิดอย่างไร และ เชื่อไวยากรณ์ที่มันเขียนไม่ได้

2. สิ่งที่ต้องแก้ (What we need to fix)

รากของปัญหาคือสถาปัตยกรรมปัจจุบันยึด “ความสามารถทั้งหมด” ไว้ใน **น้ำหนักของโมเดล** ทำให้เกิดข้อเสียที่แก้ไม่ได้ในตัวเอง 3 ข้อ:

อาการที่ต้องแก้	ต้นเหตุเชิงสถาปัตยกรรม
แต่งไวยากรณ์/ไค้ตผิดอย่างมั่นใจ	โมเดลถูกบังคับให้ “เดา” โครงสร้างที่เข้มงวด (ไวยากรณ์) จากปริภูมิแฝงที่สูญเสียข้อมูล
อธิบายวิธีคิดไม่ได้ (กล่องดำ)	เหตุผลอยู่ในเวกเตอร์แฝง อ่านไม่ได้ ตรวจสอบย้อนหลังไม่ได้
เก่งขึ้น = ต้องฝึกซ้ำ/ขึ้นคลาวด์	ความรู้ผูกกับน้ำหนัก การอัปเดตต้อง fine-tune (พลังงานสูง + “ลืม” ของเดิม) หรือส่งข้อมูลออก

เป้าหมายของโครงการนี้จึงกลับด้าน: แยกความสามารถออกจากน้ำหนักโมเดล แล้วย้ายไปไว้ใน **กราฟความรู้ที่ตรวจสอบได้** ซึ่งเป็นข้อความที่คนอ่านได้

3. สิ่งที่ต้องการจริง ๆ (What we really need)

เครื่องมือที่ใช้ได้จริงในโรงเรียน SME และองค์กรที่หวงข้อมูล ต้องมีคุณสมบัติสี่ข้อพร้อมกัน:

¹Veracode, “2025 GenAI Code Security Report” — veracode.com/blog/ai-generated-code-security-risks/
²Stanford (Perry et al.), “Do Users Write More Insecure Code with AI Assistants?”
³Forbes/CNBC, พ.ค. 2023, “Samsung Bans ChatGPT After Sensitive Code Leak.”
⁴IEA, “Data centre electricity use surged in 2025,” [iea.org](https://www.iea.org/).

1. อยู่บนเครื่อง (on-device): โมเดลเล็ก ($\sim 3B$, 4-bit) ตรึงค่า (frozen) รันบน GPU โน้ตบุ๊ก ≤ 6 GB ออฟไลน์ ต้นทุนต่อครั้ง = 0
2. ถูกต้องโดยการตรวจสอบ (verified): ทุกความรู้ที่บันทึกต้อง ผ่านการทดสอบ ก่อน กราฟจึงถูกต้อง “โดยโครงสร้าง”
3. อธิบายได้อย่างซื่อสัตย์ (faithful explanation): ระบบต้องบอกได้ว่ามัน ใช้ขั้นตอนใด เพราะอะไร และคำอธิบายต้อง ตรงกับสิ่งที่รันจริง ไม่ใช่แต่ง ที่หลัง
4. เก่งขึ้นโดยไม่ต้องฝึก: “เรียนรู้” = เพิ่มโหนด/เส้นเชื่อมในกราฟ ไม่ใช่ gradient descent

กฎที่เราวัดได้ (พิสูจน์อิสระ 3 ครั้ง): โมเดลที่ตรึงค่าเป็นฟังก์ชัน ข้อความเข้า-ข้อความออก ช่องทางเดียวที่ป้อนเข้าได้แบบไม่สูญเสียคือ **ข้อความ (ไม่ใช่เวกเตอร์แฝง)** ดังนั้นระบบภายนอกช่วยได้เพียง (ก) เลือกข้อความที่ตรวจแล้ว ป้อนให้ และ (ข) ปลอ่ยให้โมเดลทำสิ่งที่ถนัด ตารางล่างคือ “แผนที่ของสิ่งที่ทำได้/ไม่ได้” ภายใต้ข้อจำกัดนี้:

ช่องทางที่ทดลองป้อนให้โมเดล	สิ่งที่โมเดลเล็กผลิต	ผล (3B จริง)
เวกเตอร์แฝง (latent)	เวกเตอร์ให้ LM อ่าน	ทำไม่ได้ (routing collapse 73 $\rightarrow$ 15)
ร่างโค้ดแทน (draft)	โทเคนโค้ด	ทำไม่ได้ (0/3 เป็นขยะ)
โครงสร้าง: เลือกอะตอมใด (route)	ตัวชี้ (pointer)	ทำได้ (cosine 0.50 $\rightarrow$ 0.98)
โครงสร้าง: ต่ออะตอมอย่างไร (plan)	โปรแกรมอะตอม	ทำได้ (0.03 $\rightarrow$ 1.00)

Table 1: ทุกช่องทางที่ “ไม่ใช่โครงสร้าง” ล้มเหลว ทั้งสองช่องทาง “โครงสร้าง” สำเร็จ — นี่คือเหตุผลของสถาปัตยกรรม

4. ผลงานของเรา: สถาปัตยกรรมสองช่องสัญญาณ (Show our work)

เรานำกฎข้างต้นมาสร้างระบบที่แยกความหมาย (การอธิบาย) ออกจากไวยากรณ์ (การเขียนโค้ดที่เป๊ะ) เป็นสองช่องสัญญาณ: ช่องสัญญาณ สังเกตโครงสร้างโค้ดที่ตรวจแล้วจากกราฟ (แม่นยำ ไม่มีทางแต่งผิด) ส่วน ช่องความหมาย ให้ LM เล่าอธิบายจากการเดินกราฟจริง (ตรงกับที่รัน)

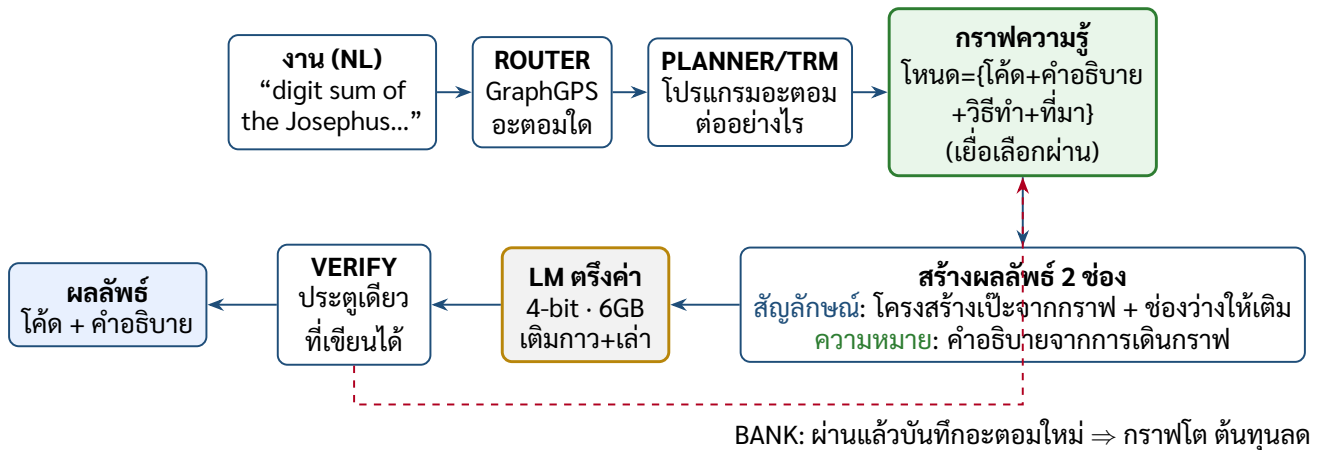


Figure 1: ท่อการทำงาน (pipeline): ROUTER เลือกอะตอม → PLANNER/TRM วางโครงสร้าง → กราฟส่งโค้ดที่ตรวจแล้วเป็น “ข้อความ” → สร้างผลลัพธ์สองช่อง (สัญญาณ=เป๊ะ, ความหมาย=อธิบาย) → **VERIFY เป็นประตูดียวที่เขียนกราฟได้** → ผ่านแล้ว BANK ทำให้กราฟโตและต้นทุน LM ต่อครั้งลดลง (compounding) โมเดลไม่เคยถูกฝึก

4.1 ตัวอย่างการทำงานจริง 1 ครั้ง (วางคู่ขนานกับท่อ)

ตารางล่างคือ “การรันจริงหนึ่งครั้ง” ของงาน “ผลรวมหลักของตำแหน่งผู้รอดชีวิต Josephus” ไล่ทีละสถานี ให้เห็นว่าแต่ละกล่องในรูปที่ 1 ผลิตอะไรออกมาจริง:

สถานีในท่อ	ผลลัพธ์จริงของตัวอย่างนี้
1. งาน (NL)	“the digit sum of the Josephus survivor position for n”
2. ROUTER	จัดอันดับอะตอมด้วยความหมาย → เลือก josephus, digit_sum
3. PLANNER/ TRM	โปรแกรมอะตอม: digit_sum(josephus(n)) (ต่อสองอะตอม)
4. กราฟ (สัญลักษณ์)	ดึง ขอร์สที่ตรวจแล้ว ของ josephus และ digit_sum มาเป็นโครงสร้างที่ “ล็อก” (LM แก้มไม่ได้)
5. LM (ช่องความหมาย)	เติมการ return digit_sum(josephus(n)) + เล่า: “ใช้ josephus หาตำแหน่งผู้รอด แล้ว digit_sum รวมหลัก”
6. VERIFY	รันกับชุดทดสอบจริง → ผ่าน (closure เป๊ะ 100%)
7. BANK	บันทึกกลับ กราฟไต่ขึ้น งานถัดไปที่ใช้ josephus ไม่ต้องเรียก LM อีก

Table 2: การรันจริงหนึ่งครั้ง: ผลลัพธ์เป็นของโมเดลเอง (อธิบายได้) แต่โครงสร้างที่ยากมาจากกราฟ (ไม่มีทางแต่งไวยากรณ์ผิด) — “ความหมายเป็นของ LM ไวยากรณ์เป็นของกราฟ”

4.2 ผลการทดลอง (วัดจริงบน 3B, ฮาร์ดแวร์เดียว)

สิ่งที่วัด (Qwen2.5-3B จริง)	ไม่ใช้วิธีเรา	ใช้วิธีเรา
สองช่องสัญญาณ: งานที่ “ผ่านการทดสอบ” ($n=40$)	18/40 (เขียนอิสระ)	39/40
ไวยากรณ์ผิดที่หลุดออกไป	5/40	0/40
ความตรงของคำอธิบายกับโค้ดจริง (faithfulness)	—	1.00
ความจำ+ผู้ให้เหตุผล vs RAG: งานที่ไม่เคยเห็น ($n=40$)	15/40 (RAG)	31/40
จำนวนครั้งที่เรียก LM (ยิ่งน้อยยิ่งดี)	100	63
เพดานการ “ประกอบ” (ความลึก 5)	0.03	1.00
การค้นอะตอมด้วยโครงสร้าง (เทียบ cosine)	0.50	0.98

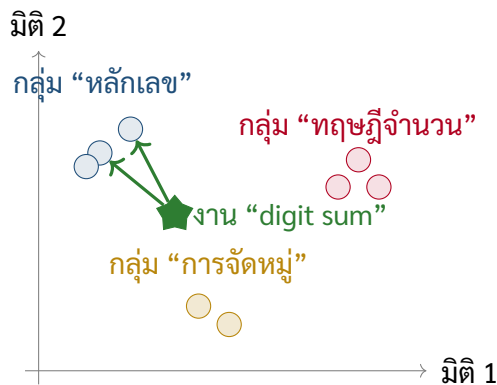
Table 3: ทุกตัวเลขวัดจากการรันจริงบนโมเดล 3B ตรึงค่า “ผ่านการทดสอบ” = ชุดทดสอบรันผ่านจริง ไม่ใช่แค่คอมไพล์ได้

อ่านผล: (1) ช่องสัญญาณลักษณะทำให้ **ไวยากรณ์ผิดเป็นศูนย์** เพราะโครงสร้างยากมาจากกราฟ ไม่ใช่การเดาของโมเดล — ตอบโจทย์ “เชื่อถือไม่ได้” ในหัวข้อ 1 โดยตรง (2) กราฟที่เติบโตได้ **ชนะ RAG 31 ต่อ 15** พร้อม เรียก LM น้อยกว่า เพราะนำอะตอมที่ตรวจแล้วกลับมาใช้ (compounding: derived-reuse วัดได้ 6→70 บนสตรีมจริง) (3) เมื่อให้ “โครงสร้าง” เพดานการประกอบที่โมเดลเปล่าทำได้ 3% กลายเป็น 100%

5. ผู้ให้เหตุผล⁺จิ๋ว (TRM) และปริภูมิความหมายของการค้น

หัวใจการ “เลือกอะตอมและวางโครงสร้าง” คือโมเดลจิ๋วสองส่วน: **ตัวจัดเส้นทาง (GraphGPS)** ที่ฝังงานและอะตอมลงในปริภูมิความหมายเดียวกันแล้ววัดความใกล้ + เดินตามเส้นเชื่อม และ **ผู้ให้เหตุผล TRM** (พารามิเตอร์หลักหมื่น) ที่เขียนสถานะย่อยที่ตรวจแล้วลง “ความจำที่ไม่จางหาย” รูปซ้ายคือภาพปริภูมิฝัง: อะตอมจับกลุ่มตามความหมาย และงานใหม่ถูกส่งไปกลุ่มที่ถูกต้อง (แก้ปัญห

“digits” vs “divisors” ที่การจับคู่คำทำไม่ได้)



ความจำที่ไม่อาจหายของ TRM (วัดจริง): ฝึกบนช่องว่างสั้น (≤ 20) ทดสอบบนช่องว่างยาวที่ไม่เคยเห็น

ช่องว่าง	GRU	TRM
20 (สุดที่ฝึก)	0.45	0.95
100 ($5\times$)	0.50	0.98

TRM “เรียนกลไก” ไม่ใช่ท่องความยาว: แม่นคงที่ ~ 0.97 เมื่อยืดไกล 5 เท่าของช่วงฝึก

Figure 2: ซ้าย: ปริภูมิฝังของการค้น — อะตอมจับกลุ่มตามความหมาย งานถูกส่งไปกลุ่มที่ถูก ขวา: TRM เรียนรู้การจำที่ไม่อาจและ วางนัยทั่วไปเกินช่วงฝึก

6. การขยายสู่การใช้งานจริง (Deployment scaling)

- **ครุตรวจแทนการฝึก:** ตอนพัฒนา ใช้ “โมเดลครู” ขนาดใหญ่ (นอกเครื่อง) ช่วยตัดสินเฉพาะที่ไม่มีเครื่องตรวจอัตโนมัติ แต่ เครื่องตรวจจริง (รันโค้ด) ยังเป็นหลัก — ครูเสนอ ความจริงมาจากการตรวจ
- **ปรับจูน TRM+GPS ด้วย RL โดยไม่ให้ “ยวบตัว”:** การฝึกประตูเลือกช่อง (gate) แบบธรรมดาจะยวบเป็น “เขียนโค้ดอย่างเดียว ไม่อธิบาย” หรือ “อธิบายอย่างเดียว หลอนโค้ด” เราออกแบบ ฟังก์ชันรางวัลผสม (ผลรันจริง + รางวัลอธิบายที่วัด “ความตรง” + โทษใช้ผิดช่อง + เอนโทรปี) และหลักสูตรฝึกเป็นขั้น พิสูจน์แล้วแบบไม่ใช้ GPU ว่าเข้าสู่จุดสมดุลสองช่องได้ (โครงสร้าง→กราฟ 0%, อธิบาย→LM 100%, ผ่าน 100%) และเห็นทั้งสองแบบ “ยวบตัว” เมื่อถอดองค์ประกอบออก
- **TRM ปรับตัวได้ แต่ต้องผ่านประตูตรวจสอบ:** LM ตรึงตลอดไป ส่วน TRM อัปเดตได้ เฉพาะจากร่องรอยที่ผ่านการตรวจแล้ว (ไม่ใช่เรียนรู้ออนไลน์ดิบๆ ที่ทำให้ “เลือนลอย”) ค่าเริ่มต้นบนเครื่องคือ TRM ตรึง ให้ “กราฟ” เป็น

ตัวปรับตัว ครูฝึก TRM ใหม่ นอกเครื่องแล้วส่งอัปเดต

7. ข้อจำกัด (ระบุตรงไปตรงมา)

- เพดานคือ “การประดิษฐ์” อะตอมใหม่: กราฟนำอะตอมที่รู้แล้วมาใช้/ต่อ ถ้างานต้องการพื้นฐานใหม่ที่ 3B เขียนเองไม่ได้ ก็ยังเป็นเพดาน — การให้ LM ประดิษฐ์อะตอมใหม่คือแนวหน้าถัดไป
- ขอบเขตอยู่ที่โดเมนที่ตรวจได้: การเติบโตที่ปลอดภัยต้องมีเครื่องตรวจ (โค้ด/คณิต/จำลอง) โดเมนปลายเปิดยังต้องพึ่งการค้นคืน — เราระบุขอบเขตนี้ ไม่กล่าวเกินจริง
- ตัวอย่างยังเล็ก: ตัวเลขวัดบนชุดหลักสิบ เป็นสัญญาณเชิงบวกและกลไกที่พิสูจน์ได้ ไม่ใช่การอ้างว่าดีที่สุด กำลังขยายไปชุดทดสอบยากขึ้น (MHPP, BigCodeBench)

8. สรุป

โครงการนี้เปลี่ยน “การทำให้ AI เก่งขึ้น” จากการฝึกน้ำหนักที่กินพลังงานและตรวจไม่ได้ ให้เป็น การแก้กราฟที่ถูก โปร่งใส และตรวจสอบได้ บนโมเดลเล็ก ตรึงค่าที่รันออฟไลน์บนเครื่องราคาประหยัด สถาปัตยกรรมสองช่องสัญญาณทำให้ผลลัพธ์ เป็นของโมเดลเอง อธิบายได้ และไวยากรณ์ผิดเป็นศูนย์ (39/40 เทียบ 18/40 บน 3B จริง) กราฟที่เติบโตได้ชนะ RAG พร้อมเรียกโมเดลน้อยลง และโครงสร้างของกราฟลบเพดานการประกอบจาก 3% เป็น 100% — ตอบทั้งสามปัญหาจริงในหัวข้อ 1 (เชื่อถือได้ เป็นส่วนตัว ประหยัดพลังงาน) ด้วยหลักฐานที่วัดได้จริง